

Training Data

```
In [ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import OneHotEncoder #
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, recall_score, precision_score, confusion_matrix
```

Loading training data. We split our training data into two parts. First 400 are used as training data, next 150 are used as validation data.

```
In [ ]: train_data = pd.read_csv('train_data.csv', index_col=0)

train_data.loc[train_data['OCCUPATION']!=1, 'OCCUPATION'] = 0 #
tr_data = train_data[0:400] #
va_data = train_data[400:] #
va_data.index = range(150)
```

```
In [ ]: train_data
```

Prepping data

```
In [ ]: # first, we list all the categorical variables to be one hot encoded
cat_vars = ['MARRIAGE', 'EDUCATION']
```

```
In [ ]: encoders = [OneHotEncoder(categories='auto') for _ in range(len(cat_vars))] #

#
encoded_tr = [encoders[i].fit_transform(tr_data[[cat_var]]).todense() for i,cat_var in enumerate(cat_vars)]
encoded_va = [encoders[i].transform(va_data[[cat_var]]).todense() for i,cat_var in enumerate(cat_vars)]
```

```
In [ ]: # aggregating our data with the one hot encoded data
# drop the label column and also drop the cat_vars
# this way we can join the encoded categorical variables with the continuous variables
X_train = pd.concat([tr_data.iloc[:, :-1].drop(cat_vars, axis=1),
                    pd.DataFrame(np.concatenate(encoded_tr, axis=1)), axis=1]
X_validation = pd.concat([va_data.iloc[:, :-1].drop(cat_vars, axis=1),
                          pd.DataFrame(np.concatenate(encoded_va, axis=1)), axis=1]
y_train = tr_data.iloc[:, -1]
y_validation = va_data.iloc[:, -1]
X_train = X_train.rename(columns={0: 'Marriage 1', 1: 'Marriage 2', 2: 'Marriage 3', 3: 'Edu 1', 4: 'Edu 2', 5: 'Edu 3',
                                   6: 'Edu 4', 7: 'Edu 5', 8: 'Edu 6', 9: 'Edu 7'})
X_validation = X_validation.rename(columns={0: 'Marriage 1', 1: 'Marriage 2', 2: 'Marriage 3', 3: 'Edu 1', 4: 'Edu 2',
                                             5: 'Edu 3',
                                             6: 'Edu 4', 7: 'Edu 5', 8: 'Edu 6', 9: 'Edu 7'})
```

```
In [ ]: X_train.head()
```

```
In [ ]: #
for i in [0,1,2,3,4,5,8]:
    X1 = X_train.iloc[:, i]
    mean = X1.mean()
    std = X1.std()
    X_train.iloc[:, i] = (X1-mean)/std
    X_validation.iloc[:, i] = (X_validation.iloc[:, i]-mean)/std
```

```
In [ ]: X_train.head()
```

We use the regularized logistic regression to see results

```
In [ ]: lr = LogisticRegression(penalty='l1', solver='liblinear', C=1)
lr.fit(X_train, y_train)
```

```
In [ ]: print('Intercept = ', lr.intercept_[0])
        # match coefs with categories
        Ind=[i for i,x in enumerate(lr.coef_.flatten()) if x!=0]
        dict(zip(X_train.columns[Ind],lr.coef_.flatten()[Ind]))
```

```
In [ ]: n = X_validation.shape[0]
        pred = lr.predict(X_validation) #
        TN,FP,FN,TP = confusion_matrix(y_validation, pred).ravel() #
        print('Accuracy: ', accuracy_score(y_validation, pred)) #
        print('Precision: ', precision_score(y_validation, pred)) #
        print('Recall (True Positive): ', recall_score(y_validation, pred)) #
        print('True Negative Rate: ', TN/(TN+FP)) #
        print('Sum of True Positive Rate and True Negative Rate', TP/(TP+FN)+TN/(TN+FP)) #
```

```
In [ ]: #
        Q = lr.predict_proba(X_validation)[:,:1]
        THRESHOLD = [0.4, 0.45, 0.5, 0.55, 0.6]
        for i in THRESHOLD:
            print('\nTHRESHOLD = ',i)
            pred = np.where(Q>i,1,0)
            TN,FP,FN,TP = confusion_matrix(y_validation, pred).ravel()
            print('Accuracy: ', accuracy_score(y_validation, pred))
            print('Precision: ', precision_score(y_validation, pred))
            print('Recall (True Positive): ', recall_score(y_validation, pred))
            print('True Negative Rate: ', TN/(TN+FP))
            print('Sum of True Positive Rate and True Negative Rate', TP/(TP+FN)+TN/(TN+FP))
```

```
In [ ]: from sklearn.model_selection import GridSearchCV #
```

```
In [ ]: # Try L2 regularization instead of L1 regularization
        lr = LogisticRegression(penalty='l2',solver='liblinear') # L2 regularization parameter
        params = {'C':[i/100 for i in range(1,101)]} #
```

The following is our grid search function, searching through all parameters in param. It looks for the parameters that maximizes accuracy, precision, and recall. Since there are three things to improve on, we pick recall as the one that determines the best parameter. (This is an arbitrary choice)

```
In [ ]: clf = GridSearchCV(lr, params, cv=5, scoring='recall')
```

```
In [ ]: clf.fit(X_train, y_train)
```

```
In [ ]: clf.best_params_ #
```

```
In [ ]: lr = LogisticRegression(penalty='l2', solver='liblinear', C=0.8) #  
lr.fit(X_train, y_train) #  
pred = lr.predict(X_validation) #  
TN, FP, FN, TP = confusion_matrix(y_validation, pred).ravel()  
print('Accuracy: ', accuracy_score(y_validation, pred))  
print('Precision: ', precision_score(y_validation, pred))  
print('Recall (True Positive): ', recall_score(y_validation, pred))  
print('True Negative Rate: ', TN/(TN+FP))  
print('Sum of True Positive Rate and True Negative Rate', TP/(TP+FN)+TN/(TN+FP))
```

```
In [ ]: #  
import pickle  
filename = 'finalized_model.sav'  
pickle.dump(lr, open(filename, 'wb'))
```

Test the Model Out-of-sample

```
In [ ]: # delete all variables and functions in namespace  
%reset  
import numpy as np  
import pandas as pd  
from sklearn.preprocessing import OneHotEncoder  
from sklearn.preprocessing import LabelEncoder  
from sklearn.linear_model import LogisticRegression  
from sklearn.metrics import accuracy_score, recall_score, precision_score, confusion_matrix
```

```
In [ ]: #
test_data = pd.read_csv('test_data.csv',index_col=0)
train_data=pd.read_csv('train_data.csv',index_col=0)

# Convert "Occupation" into a dummy variable
test_data.loc[test_data['OCCUPATION']!=1,'OCCUPATION'] = 0
train_data.loc[train_data['OCCUPATION']!=1,'OCCUPATION'] = 0

total_data=pd.concat([train_data,test_data])
total_data=total_data.reset_index(drop=True)
```

```
In [ ]: # first, we list all the categorical variables to be one hot encoded
cat_vars = ['MARRIAGE', 'EDUCATION']
```

```
In [ ]: encoders = [OneHotEncoder(categories='auto') for _ in range(len(cat_vars))] # create an encoder for each cat_
vars

# encode each of the cat_vars with their respective encoder
encoded_total = [encoders[i].fit_transform(total_data[[cat_var]]).todense() for i,cat_var in enumerate(cat_va
rs)]
```

```
In [ ]: # aggregating our data with the one hot encoded data
# drop the label column and also drop the cat_vars
# this way we can join the encoded categorical variables with the continuous variables
X_total = pd.concat([total_data.iloc[:, :-1].drop(cat_vars, axis=1),
                    pd.DataFrame(np.concatenate(encoded_total, axis=1)), axis=1)

y_test = test_data.iloc[:, -1]
X_test=X_total.tail(len(y_test))
X_test= X_test.rename(columns={0:'Marriage 1',1:'Marriage 2',2:'Marriage 3',3:'Edu 1',4:'Edu 2',5:'Edu 3',
6:'Edu 4',7:'Edu 5',8:'Edu 6',9:'Edu 7'})

X_train=X_total[0:len(X_total)-len(y_test)]
```

```
In [ ]: #
        for i in [0,1,2,3,4,5,8]:
            X1 = X_train.iloc[:,i]
            mean = X1.mean()
            std = X1.std()
            X_test.iloc[:,i] = (X_test.iloc[:,i]-mean)/std
```

```
In [ ]: X_test.head()
```

```
In [ ]: #
        import pickle
        lr_load = pickle.load(open('finalized_model.sav', 'rb'))
```

```
In [ ]: #
        pred = lr_load.predict(X_test)
        TN,FP,FN,TP = confusion_matrix(y_test, pred).ravel()
        print('Accuracy: ', accuracy_score(y_test, pred))
        print('Precision: ', precision_score(y_test, pred))
        print('Recall (True Positive): ', recall_score(y_test, pred))
        print('True Negative Rate: ',TN/(TN+FP))
        print('Sum of True Positive Rate and True Negative Rate',TP/(TP+FN)+TN/(TN+FP))
```